

你好，朱峰正同学！欢迎你来学习今天的课程。我是你的数字取证课程讲师。

今天我们要深入探讨的主题是 **取证中的镜像技术**。这对于教育行业来说尤其重要，因为学校和教育机构经常需要处理大量的学生数据、教学资料和行政文件，一旦发生数据安全事件或需要内部审查，规范、可信的电子数据取证就是关键。

考虑到你之前的基础，我会把今天的90分钟内容讲得非常细致，确保每一步你都能跟上。我们会从一个教育行业的典型场景出发，手把手地完成一次完整的取证镜像实践。

1. 谁想做什么？遇到了什么痛点？

想象一下，你是一所大学的网络中心主任。学校接到匿名举报，称某位教务管理人员可能利用职务之便，篡改了学生的成绩数据。校领导要求你立即对这位管理人员使用的办公电脑进行电子数据取证，以查明真相。

你面临的痛点是什么？

- **数据量大且敏感**：电脑里存有全校数万学生的成绩记录，数据量巨大，且必须保密。
- **证据易被破坏**：如果你直接打开电脑检查，任何操作（比如不小心修改了文件时间）都可能破坏原始证据的真实性，导致结果在法律上无效。
- **需要可复现性**：调查过程必须能被第三方机构（例如司法鉴定中心）重复验证，确保结果公正。

旧方法为何无效？

过去，有人可能会直接登录系统，把可疑文件复制出来分析。这种方法的问题是：

- **改变了证据**：复制操作会改变文件的“最后访问时间”等元数据，破坏了原始状态。
- **无法证明完整性**：你怎么向别人证明，你分析的文件和原始硬盘上的文件一模一样，没有被你无意中修改过？
- **容易遗漏**：手动复制可能会漏掉隐藏文件、磁盘碎片中的残留数据等关键证据。

案例对比：这就好比警察调查一个传统案件的物证（比如一把刀）。正确的做法是戴着手套，将物证放入证物袋密封，贴上标签，记录所有经手人。而错误的做法就是直接用手拿着刀翻来覆去地看，这样会在上面留下自己的指纹，破坏了证据。

2. 新解决方案的核心：取证镜像

为了解决上述痛点，数字取证领域引入了 **镜像技术**。它的核心思想非常简单却极其重要：

为原始数据制作一个完美的、位对位的“数字副本”（即镜像），所有后续的分析工作都只在这个副本上进行。

核心差异与适用条件

- **核心差异**：镜像技术 **不关心** 文件的具体内容（比如它是Word文档还是图片），它只关心存储介质上最底层的“0”和“1”的排列顺序。它会把整个硬盘扇区一个接一个地、原封不动地复制下来，包括已删除文件的空间、系统空闲区域等。而普通复制只复制当前可见的文件。
- **适用条件**：只要是需要确保电子证据 **完整性、真实性和不可否认性** 的场景，都必须使用镜像技术。这在教育、金融、司法等领域是基本要求。

一个简单的比喻

你可以把硬盘想象成一盘珍贵的、不能重复录制的录像带。取证镜像就像是使用一台专业的克隆机，将这盘录像带 **连同所有内容（包括空白间隙）** 完整地复制到另一盘空白带上。以后我们所有的“播放”和“分析”都只用那盘复制带，原始录像带则被安全地锁进保险柜。

3. 实战演练：为教育行业场景构建取证镜像环境

现在，我们开始动手。假设我们要调查的是一台运行Linux系统的教务管理服务器。

第一步：数据准备与环境隔离

1. **目标设备**：假设目标硬盘是 `/dev/sdb`（**非常重要：操作前务必确认设备名，误操作会导致数据丢失！**）。
2. **操作机**：准备另一台干净的、安装有Linux系统的电脑作为我们的取证工作机。
3. **存储介质**：准备一个容量 **大于** 目标硬盘的空白移动硬盘，用于存放镜像文件，我们将其挂载到 `/mnt/evidence`。
4. 由于这里没有硬盘，所以选择使用U盘代替，由下图可见处于sdb/sdb1

```
(kali@20232311zfz)-[~]
$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0 80.1G  0 disk
└─sda1       8:1    0 80.1G  0 part /
sdb          8:16    1 57.7G  0 disk
├─sdb1       8:17    1 57.1G  0 part
└─sdb2       8:18    1  297M  0 part
sr0         11:0    1 1024M  0 rom
sr1         11:1    1  53.6M  0 rom
```

```
(kali@20232311zfz)-[~]
$ sudo umount /dev/sdb*
[sudo] kali 的密码:
umount: /dev/sdb: not mounted.
umount: /dev/sdb1: not mounted.
umount: /dev/sdb2: not mounted.
```

```
(kali@20232311zfz)-[~]
$ sudo mkdir -p /mnt/evidence
```

第二步：安装必要工具

在取证工作机上，打开终端，安装核心工具：

```
# 更新软件包列表
sudo apt-get update

# 安装取证工具包（包含dd、dcfldd等）、哈希计算工具、自动取证分析工具
sudo apt-get install dcfldd hashdeep sleuthkit autopsy -y
```

```
(kali@20232311zfz)-[~]
$ sudo apt-get update
获取:1 http://mirrors.ustc.edu.cn/kali kali-rolling InRelease [34.0 kB]
获取:2 http://mirrors.ustc.edu.cn/kali kali-rolling/main i386 Packages [20.5 MB]
获取:3 http://mirrors.ustc.edu.cn/kali kali-rolling/main amd64 Packages [21.0 MB]
获取:4 http://mirrors.ustc.edu.cn/kali kali-rolling/main amd64 Contents (deb) [53.5 MB]
获取:5 http://mirrors.ustc.edu.cn/kali kali-rolling/main i386 Contents (deb) [48.4 MB]
获取:6 http://mirrors.ustc.edu.cn/kali kali-rolling/contrib amd64 Packages [118 kB]
获取:7 http://mirrors.ustc.edu.cn/kali kali-rolling/contrib i386 Packages [100 kB]
获取:8 http://mirrors.ustc.edu.cn/kali kali-rolling/contrib amd64 Contents (deb) [275 kB]
获取:9 http://mirrors.ustc.edu.cn/kali kali-rolling/contrib i386 Contents (deb) [193 kB]
获取:10 http://mirrors.ustc.edu.cn/kali kali-rolling/non-free amd64 Packages [186 kB]
获取:11 http://mirrors.ustc.edu.cn/kali kali-rolling/non-free i386 Packages [141 kB]
获取:12 http://mirrors.ustc.edu.cn/kali kali-rolling/non-free i386 Contents (deb) [860 kB]
获取:13 http://mirrors.ustc.edu.cn/kali kali-rolling/non-free amd64 Contents (deb) [892 kB]
获取:14 http://mirrors.ustc.edu.cn/kali kali-rolling/non-free-firmware i386 Packages [13.8 kB]
获取:15 http://mirrors.ustc.edu.cn/kali kali-rolling/non-free-firmware amd64 Packages [14.3 kB]
获取:16 http://mirrors.ustc.edu.cn/kali kali-rolling/non-free-firmware amd64 Contents (deb) [33.8 kB]
获取:17 http://mirrors.ustc.edu.cn/kali kali-rolling/non-free-firmware i386 Contents (deb) [33.8 kB]
已下载 146 MB, 耗时 1分 45秒 (1,398 kB/s)
正在读取软件包列表 ... 完成

(kali@20232311zfz)-[~]
$ sudo apt-get install dcfldd hashdeep sleuthkit autopsy -y
正在读取软件包列表 ... 完成
正在分析软件包的依赖关系树 ... 完成
正在读取状态信息 ... 完成
hashdeep 已经是最新版 (4.4+git20170824+ds-1).
hashdeep 已设置为手动安装。
autopsy 已经是最新版 (2.24-6kali1)。
autopsy 已设置为手动安装。
下列软件包是自动安装的并且现在不需要了:
libplacebo349 libravie0.7 libtheoraenc1 libtsk19t64 libx264-164
使用'sudo apt autoremove'来卸载它(它们)。
将会同时安装下列软件:
libbfiol1 libtsk23 libvslvmt64
下列【新】软件包将被安装:
dcfldd libbfiol1 libtsk23 libvslvmt64
下列软件包将被升级:
sleuthkit
升级了 1 个软件包, 新安装了 4 个软件包, 要卸载 0 个软件包, 有 2400 个软件包未被升级。
需要下载 1,460 kB 的归档。
解压后还会消耗 2,950 kB 的额外空间。
获取:1 http://mirrors.ustc.edu.cn/kali kali-rolling/main amd64 dcfldd amd64 1.9.3-2 [45.3 kB]
获取:2 http://http.kali.org/kali kali-rolling/main amd64 libbfiol1 amd64 20170123-6+b3 [311 kB]
获取:3 http://kali.download/kali kali-rolling/main amd64 libvslvmt64 amd64 20240504-2 [422 kB]
获取:4 http://http.kali.org/kali kali-rolling/main amd64 sleuthkit amd64 4.14.0+dfsg-0kali1 [300 kB]
```

- **dcfldd** : 是 **dd** 命令的增强版, 自带哈希校验和进度显示, 更适合取证。
- **sleuthkit** / **autopsy** : 强大的开源取证分析平台, 我们后面会用来自动分析镜像。

第三步：法律合规要点实操（重中之重！）

在进行任何操作前, 我们必须遵守法律合规流程：

1. 只读原则（软件写保护）：

在将目标硬盘连接到取证工作机后, **立即** 执行以下命令, 将其设置为只读模式, 防止误写操作。

```
sudo blockdev --setro /dev/sdb
验证是否设置成功:
sudo blockdev --getro /dev/sdb
如果返回 `1`, 则表示成功设置为只读。
```

```
(kali@20232311zfz)-[~]
$ sudo blockdev --setro /dev/sdb

(kali@20232311zfz)-[~]
$ sudo blockdev --getro /dev/sdb
1
```

2. 开始制作镜像并记录证据链

A. 磁盘全盘镜像（最核心步骤）

我们使用 `\`dcfldd\`` 命令制作全盘镜像, 并在过程中计算哈希值（证据链的关键）。根据国家密码管理局的要求, 我们使用商用密码算法SM3来计算哈希值。

```
# 切换到证据存储目录
cd /mnt/evidence
# 开始制作镜像
sudo dcflddd if=/dev/sdb of=./server_disk_image.img bs=4K hash=sm3
hashwindow=1G hashlog=./sm3_hashes.txt
```

```
(kali@kali-20232311zfz)~$ cd /mnt/evidence
(kali@kali-20232311zfz)~/mnt/evidence$ sudo dcflddd if=/dev/sdb of=./server_disk_image.img bs=4K hash=sm3 hashwindow=1G hashlog=./sm3_hashes.txt
dcflddd: invalid number 4Kinvalid hash: sm3
Try 'dcflddd --help' for more information.
(kali@kali-20232311zfz)~/mnt/evidence$ sudo dcflddd if=/dev/sdb of=./server_disk_image.img bs=4k hash=SM3 hashwindow=1G hashlog=./sm3_hashes.txt
invalid hash: SM3
Try 'dcflddd --help' for more information.
(kali@kali-20232311zfz)~/mnt/evidence$ sudo dcflddd if=/dev/sdb of=./server_disk_image.img bs=4k hash=sha256 hashwindow=1G hashlog=./sha256_hashes.txt
322560 blocks (1260Mb) written.
```

如图所示，由于发现不支持SM3，所以这里先采用sha256代替。

```
sudo dcflddd if=/dev/sdb of=./server_disk_image.img bs=4k hash=sha256 hashwindow=1G
hashlog=./sha256_hashes.txt
```

- * \if=/dev/sdb\`：输入文件 (Input File)，即我们的目标硬盘。
- * \of=...img\`：输出文件 (Output File)，即生成的镜像文件。
- * \bs=4K\`：设置块大小，4K通常效率较高。
- * \hash=sha256\`：指定使用sha256算法进行哈希计算。
- * \hashwindow=1G\`：每复制1GB数据就计算并记录一次哈希值。
- * \hashlog=...txt\`：将所有的哈希值记录到指定文件。

这个过程会比较长，耐心等待。完成后，\dcflddd\` 会给出整个镜像的最终sha256哈希值。

由于内存空间不足而中断

```
(kali@kali-20232311zfz)~/mnt/evidence$ sudo dcflddd if=/dev/sdb of=./server_disk_image.img bs=4k hash=sha256 hashwindow=1G hashlog=./sha256_hashes.txt
10732800 blocks (41925Mb) written.dcflddd: No space left on device
(kali@kali-20232311zfz)~/mnt/evidence$ df -h /mnt/evidence
文件系统 大小 已用 可用 已用% 挂载点
/dev/sda1 79G 79G 0 100% /
(kali@kali-20232311zfz)~/mnt/evidence$ sudo rm -f server_disk_image.img sha256_hashes.txt
[sudo] kali 的密码:
(kali@kali-20232311zfz)~/mnt/evidence$ df -h /mnt/evidence
文件系统 大小 已用 可用 已用% 挂载点
/dev/sda1 79G 38G 37G 51% /
```

于是扩容之后重新前面步骤继续进行实验

```
(kali@kali-20232311zfz)~$ df -h
文件系统 大小 已用 可用 已用% 挂载点
udev 1.9G 0 1.9G 0% /dev
tmpfs 392M 1.3M 391M 1% /run
/dev/sda1 108G 38G 66G 37% /
tmpfs 2.0G 4.0K 2.0G 1% /dev/shm
tmpfs 1.0M 0 1.0M 0% /run/credentials/systemd-journald.service
tmpfs 2.0G 56K 2.0G 1% /tmp
tmpfs 1.0M 0 1.0M 0% /run/credentials/getty@tty1.service
tmpfs 392M 124K 392M 1% /run/user/1000
```

```
(kali@20232311zfz)-[~]
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda 8:0 0 110G 0 disk
├─sda1 8:1 0 110G 0 part /
sdb 8:16 1 57.7G 0 disk
├─sdb1 8:17 1 57.1G 0 part
├─sdb2 8:18 1 297M 0 part
sr0 11:0 1 1024M 0 rom
sr1 11:1 1 53.6M 0 rom

(kali@20232311zfz)-[~]
$ sudo umount /dev/sdb*
[sudo] kali 的密码:
umount: /dev/sdb: not mounted.
umount: /dev/sdb1: not mounted.
umount: /dev/sdb2: not mounted.

(kali@20232311zfz)-[~]
$ sudo blockdev --setro /dev/sdb

(kali@20232311zfz)-[~]
$ sudo blockdev --getro /dev/sdb
1

(kali@20232311zfz)-[~]
$ cd /mnt/evidence

(kali@20232311zfz)-[/mnt/evidence]
$ sudo dcfldd if=/dev/sdb of=./server_disk_image.img bs=4k hash=sha256 hashwindow=1G hashlog=./sha256_hashes.txt
44544 blocks (174Mb) written.
```

镜像制作完成

```
(kali@20232311zfz)-[/mnt/evidence]
$ sudo dcfldd if=/dev/sdb of=./server_disk_image.img bs=4k hash=sha256 hashwindow=1G hashlog=./sha256_hashes.txt
15134720 blocks (59120Mb) written.
15134720+1 records in
15134720+1 records out

(kali@20232311zfz)-[/mnt/evidence]
$
```

这里使用dcfldd生成镜像时，哈希算法是sha256，所以先验证dcfldd日志里的sha256和镜像文件的sha256是否一致（这是镜像完整性的核心验证）：

```
(kali@20232311zfz)-[/mnt/evidence]
$ IMG_SHA256=$(sha256sum server_disk_image.img | cut -d' ' -f1)
```

```
(kali@20232311zfz)-[/mnt/evidence]
$ tail -n 10 sha256_hashes.txt
53687091200 - 54760833024: 49bc20df15e412a64472421e13fe86ff1c5165e18b2afccf160d4dc19fe68a14
54760833024 - 55834574848: 422b23a7d58e64b922da108c9a5f787b14bc9299b5cb8761d52f77fb0b02ad23
55834574848 - 56908316672: 49bc20df15e412a64472421e13fe86ff1c5165e18b2afccf160d4dc19fe68a14
56908316672 - 57982058496: 49bc20df15e412a64472421e13fe86ff1c5165e18b2afccf160d4dc19fe68a14
57982058496 - 59055800320: 49bc20df15e412a64472421e13fe86ff1c5165e18b2afccf160d4dc19fe68a14
59055800320 - 60129542144: 49bc20df15e412a64472421e13fe86ff1c5165e18b2afccf160d4dc19fe68a14
60129542144 - 61203283968: 49bc20df15e412a64472421e13fe86ff1c5165e18b2afccf160d4dc19fe68a14
61203283968 - 61991813632: 56f3b2e070a995dbcf45374e4a5dfc96d792e6fe7ead5e6e6011627deb0a10ae
```

Total (sha256): 22aff2dc6f9449229c985abefb318fa639f7a3798acd1615a3c0f92c33aee75

```
(kali@20232311zfz)-[/mnt/evidence]
$ echo "镜像文件哈希: $IMG_SHA256"
镜像文件哈希: 22aff2dc6f9449229c985abefb318fa639f7a3798acd1615a3c0f92c33aee75
```

对比发现日志里Total (sha256)后的哈希值与镜像文件计算出的SHA256哈希完全一致，证明镜像文件完整未被篡改。

B. 目录/文件镜像（针对动态或特定数据）

对于`/proc`、`/sys`这类虚拟文件系统，或者我们只想提取某个特定目录（如`/home/registrar`教务员家目录），不能使用`dd`，而应用`tar`打包，因为它能更好地处理特殊文件和时间戳。

备份特定目录，并保留所有文件属性

```
sudo tar -czvf ./home_registrar_backup.tar.gz -C / home/registrar
```

计算备份包的SM3哈希

```
sm3sum ./home_registrar_backup.tar.gz > ./home_registrar.sm3
```


- 创建挂载目录并尝试挂载镜像，如下图所示发现提示直接用loop挂载会识别不到具体文件系统

```
(kali@20232311zfz)-[/mnt/evidence]
$ sudo mount -o ro,loop server_disk_image.img /mnt/usb_mount
mount: /mnt/usb_mount: wrong fs type, bad option, bad superblock on /dev/loop0, missing codepage or helper program, or other error.
dmesg(1) may have more information after failed mount system call.
```

- 于是先找偏移量再挂载具体分区：执行命令查看镜像里的分区结构和偏移量，用计算好的偏移量挂载U盘主要分区（img1）并验证挂载成功

```
(kali@20232311zfz)-[/mnt/evidence]
$ sudo fdisk -l server_disk_image.img
Disk server_disk_image.img: 57.73 GiB, 61991813632 bytes, 121077761 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x607cc6b1

Device            Boot      Start         End      Sectors   Size Id Type
server_disk_image.img1 *        614400    120463360    119848961    57.1G  7 HPFS/NTFS/exFAT
server_disk_image.img2          120465408    121073663      608256    297M 16 Hidden FAT16

(kali@20232311zfz)-[/mnt/evidence]
$ sudo mount -o ro,loop,offset=314572800 server_disk_image.img /mnt/usb_mount

(kali@20232311zfz)-[/mnt/evidence]
$ ls /mnt/usb_mount
AUTORUN.INF 'System Volume Information' WEPE.ICO Win11_25H2_Chinese_Simplified_x64.iso
```

- 确认ISO文件路径，用tar打包这个ISO文件（保留文件属性，对齐实验要求）

```
(kali@20232311zfz)-[/mnt/evidence]
$ ls -lh /mnt/usb_mount/Win11_25H2_Chinese_Simplified_x64.iso
-rwxr-xr-x 1 root root 7.3G 3月13日 06:20 /mnt/usb_mount/Win11_25H2_Chinese_Simplified_x64.iso

(kali@20232311zfz)-[/mnt/evidence]
$ sudo tar -czvf ./win11_iso_backup.tar.gz -C /mnt/usb_mount Win11_25H2_Chinese_Simplified_x64.iso
Win11_25H2_Chinese_Simplified_x64.iso
```

- 计算打包文件的SHA256哈希

```
(kali@20232311zfz)-[/mnt/evidence]
$ sha256sum ./win11_iso_backup.tar.gz > ./win11_iso.sha256

(kali@20232311zfz)-[/mnt/evidence]
$ cat ./win11_iso.sha256
6ee7a122c80478dc0eed396015f26f843be2e15d0e4dc331f82e6758f4e178e5 ./win11_iso_backup.tar.gz
```

```
(kali@20232311zfz)-[/mnt/evidence]
$ ISO_SHA256=$(cat ./win11_iso.sha256 | cut -d' ' -f1)

(kali@20232311zfz)-[/mnt/evidence]
$ echo "Win11 ISO打包文件的SHA256哈希: $ISO_SHA256"
Win11 ISO打包文件的SHA256哈希: 6ee7a122c80478dc0eed396015f26f843be2e15d0e4dc331f82e6758f4e178e5
```

3. 原始证据隔离

镜像制作完成后，**立即**将原始硬盘从工作机上卸载，并贴上标签，注明时间、操作人、案件编号，然后进行物理封存（如放入防静电袋密封）。后续所有分析绝不再接触原始硬盘。

4. 填写取证报告 (Acquisition_Report.txt)

这是证据链的书面体现。立即创建一个报告文件，记录所有关键信息。

```
cat > /mnt/evidence/Acquisition_Report.txt << EOF
电子数据取证镜像报告
=====
案件编号: EDU-2023-001
取证目标: WinPE启动U盘 (/dev/sdb)
操作日期: $(date)
操作人员: 朱峰正
取证工具: dcfldd, tar, sha256sum
```

镜像过程记录：

- 全盘镜像文件： server_disk_image.img
- 全盘镜像SHA256哈希值： \$(grep "Total (sha256)" sha256_hashes.txt | cut -d':' -f2 | xargs)
- 目录/文件备份： win11_iso_backup.tar.gz
- 文件备份SHA256哈希值： \$(cat win11_iso.sha256 | cut -d' ' -f1)

证据链说明：

1. 操作前已执行 'blockdev --setro /dev/sdb' 启用U盘只读保护。
2. 镜像过程中已使用SHA256算法进行分段哈希校验，记录见 sha256_hashes.txt。
3. 原始U盘已于 \$(date) 物理隔离封存，编号为 EVIDENCE-001。

操作人签字： _____
监督人签字： _____
EOF

(kali@20232311zfz)-[/mnt/evidence]

\$ cat > /mnt/evidence/Acquisition_Report.txt << EOF

电子数据取证镜像报告

=====

案件编号： EDU-2023-001
取证目标： WinPE启动U盘 (/dev/sdb)
操作日期： \$(date)
操作人员： 朱峰正
取证工具： dcfldd, tar, sha256sum

镜像过程记录：

- 全盘镜像文件： server_disk_image.img
- 全盘镜像SHA256哈希值： \$(grep "Total (sha256)" sha256_hashes.txt | cut -d':' -f2 | xargs)
- 目录/文件备份： win11_iso_backup.tar.gz
- 文件备份SHA256哈希值： \$(cat win11_iso.sha256 | cut -d' ' -f1)

证据链说明：

1. 操作前已执行 'blockdev --setro /dev/sdb' 启用U盘只读保护。
2. 镜像过程中已使用SHA256算法进行分段哈希校验，记录见 sha256_hashes.txt。
3. 原始U盘已于 \$(date) 物理隔离封存，编号为 EVIDENCE-001。

操作人签字： _____
监督人签字： _____
EOF

(kali@20232311zfz)-[/mnt/evidence]

\$ cat /mnt/evidence/Acquisition_Report.txt

电子数据取证镜像报告

=====

案件编号： EDU-2023-001
取证目标： WinPE启动U盘 (/dev/sdb)
操作日期： 2026年 03月 30日 星期一 06:05:08 EDT
操作人员： 朱峰正
取证工具： dcfldd, tar, sha256sum

镜像过程记录：

- 全盘镜像文件： server_disk_image.img
- 全盘镜像SHA256哈希值： 22aff2dc6f9449229c985abefba318fa639f7a3798acd1615a3c0f92c33aee75
- 目录/文件备份： win11_iso_backup.tar.gz
- 文件备份SHA256哈希值： 6ee7a122c80478dc0eed396015f26f843be2e15d0e4dc331f82e6758f4e178e5

证据链说明：

1. 操作前已执行 'blockdev --setro /dev/sdb' 启用U盘只读保护。
2. 镜像过程中已使用SHA256算法进行分段哈希校验，记录见 sha256_hashes.txt。
3. 原始U盘已于 2026年 03月 30日 星期一 06:05:08 EDT 物理隔离封存，编号为 EVIDENCE-001。

操作人签字： _____
监督人签字： _____

第四步：使用Autopsy分析镜像

7 / 12

现在，我们开始安全地分析镜像副本。

1. 启动Autopsy（可以在图形界面或终端中运行 `autopsy` 命令）。

```
(kali㉿20232311zfz)-[/mnt/evidence]
$ sudo autopsy
[sudo] kali 的密码 :

Autopsy Forensic Browser
http://www.sleuthkit.org/autopsy/
ver 2.24

Evidence Locker: /var/lib/autopsy
Start Time: Mon Mar 30 06:08:35 2026
Remote Host: localhost
Local Port: 9999

Open an HTML browser on the remote host and paste this URL in it:

http://localhost:9999/autopsy

Keep this process running and use <ctrl-c> to exit
█
```

2. 创建一个新的案件（Case），名称如“学生成绩调查”。

CREATE A NEW CASE

1. **Case Name:** The name of this investigation. It can contain only letters, numbers, and symbols.

2. **Description:** An optional, one line description of this case.

3. **Investigator Names:** The optional names (with no spaces) of the investigators for this case.

a.

b.

c.

d.

e.

f.

g.

h.

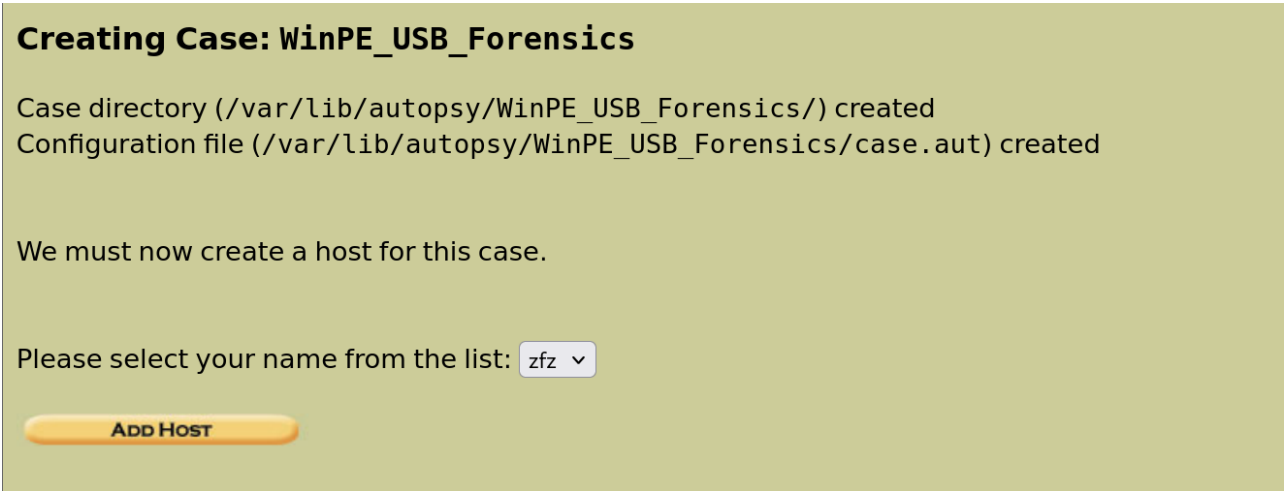
i.

j.

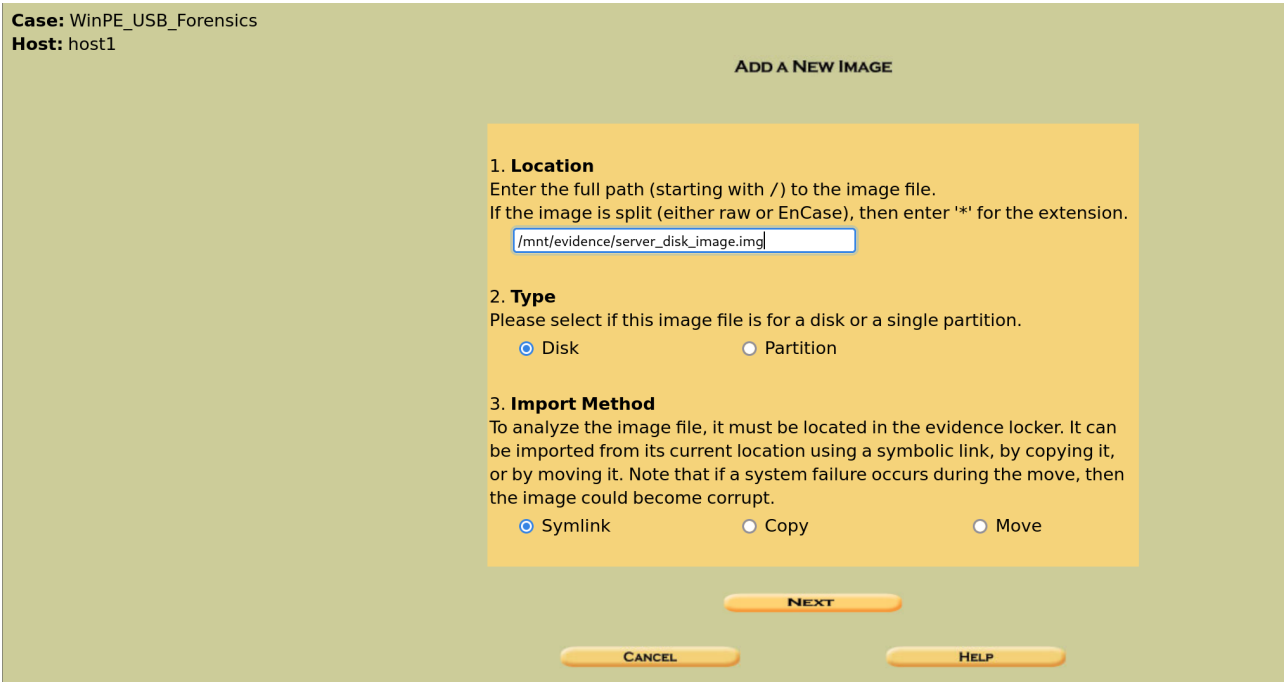
NEW CASE

CANCEL

HELP



3. 添加证据（Evidence）时，选择 **镜像文件（Image File）** ，然后找到我们生成的 `server_disk_image.img` 。



将主数据分区挂载至/、指定文件系统为fat，隐藏引导分区挂载至C:、指定为fat16，并选择忽略镜像哈希校验，加快导入流程；

Image File Details

Local Name: images/server_disk_image.img

Data Integrity: An MD5 hash can be used to verify the integrity of the image. (With split images, this hash is for the full image file)

☒ Ignore the hash value for this image.

☐ Calculate the hash value for this image.

☐ Add the following MD5 hash value for this image:

☐ Verify hash after importing?

File System Details

Analysis of the image file shows the following partitions:

Partition 1 (Type: NTFS / exFAT (0x07))

Add to case? ☒

Sector Range: 614400 to 120463360

Mount Point: / File System Type: fat

Partition 2 (Type: DOS FAT16 Hidden (0x16))

Add to case? ☒

Sector Range: 120465408 to 121073663

Mount Point: C: File System Type: fat16

ADD

CANCEL

HELP

镜像与分区已成功导入案件，系统为全盘镜像、磁盘及各卷分配唯一 ID（如img1、vol1、vol2、vol3），为后续取证分析提供标识；

Testing partitions

Linking image(s) into evidence locker

Image file added with ID img1

Disk image (type dos) added with ID vol1

Volume image (614400 to 120463360 - fat - /) added with ID vol2

Volume image (120465408 to 121073663 - fat16 - C:) added with ID vol3

OK

ADD IMAGE

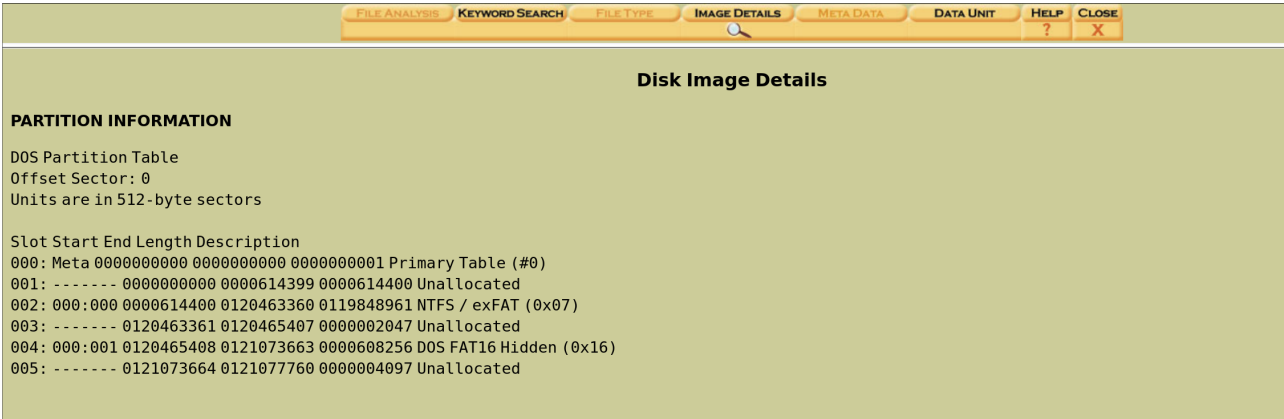
4. Autopsy会自动解析镜像中的文件系统，并允许你：
- 查看所有文件：包括已删除但未被覆盖的文件。

检索关键词：例如搜索“成绩修改”、“admin”等。

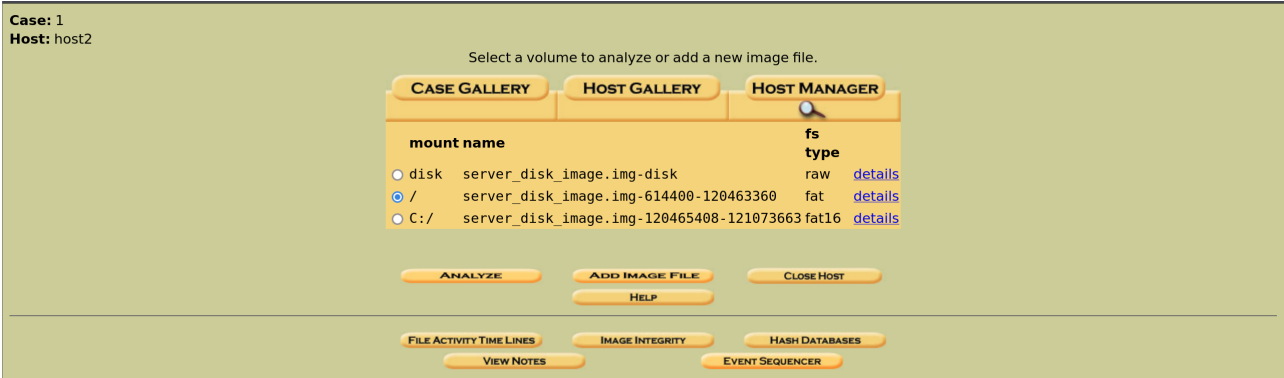
分析文件元数据：查看文件的创建、修改、访问时间线。

生成报告：Autopsy可以帮你生成详细的取证分析报告。

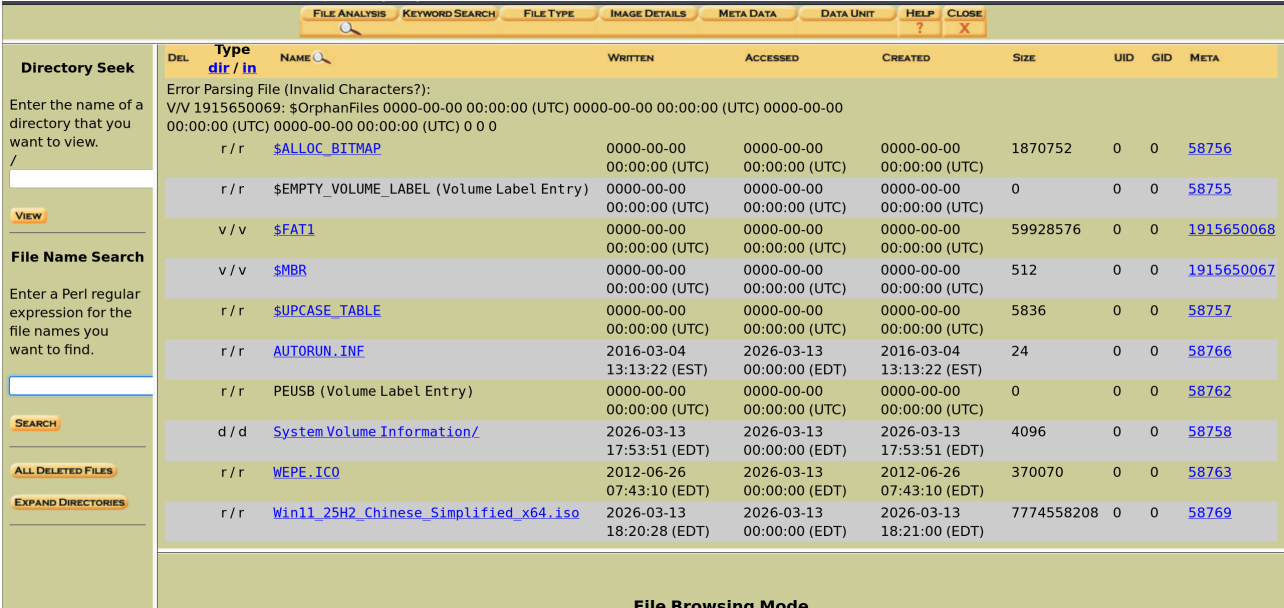
下图展示磁盘镜像的 DOS 分区表结构，确认主数据分区（NTFS/exFAT 格式）与隐藏 FAT16 引导分区的扇区范围，验证镜像完整性。
- 10 / 12



- 在案件主机界面选择挂载至/的主数据分区（FAT 格式），准备启动文件分析、时间线生成等取证操作。



- 进入主数据分区根目录/，可查看文件系统元数据（如\$FAT1）与用户文件（如 Windows 11 ISO、PE 相关文件），为后续文件检索与分析提供入口。



4. 简化的认知框架

请你记住取证镜像的三个核心原则，我把它称为“取证三不”原则：

- 1. 不直接碰原始证据（用镜像代替）。
- 2. 不破坏证据状态（操作前写保护）。
- 3. 不让证据链断裂（全程记录哈希和时间戳）。

5. 迁移到其他领域

这套方法不仅适用于教育行业。

- **企业合规审计**：调查员工是否泄露公司机密。
- **网络安全事件响应**：分析被黑客入侵的服务器。
- **个人数据恢复**：从损坏的硬盘中最大限度恢复重要数据。

6. 给你的下一步行动

今天的内容信息量很大，关键是理解和建立规范操作的意识。为了巩固学习，我建议你的下一步行动是：

在你的个人电脑上，使用虚拟机安装一个Linux系统（如Ubuntu），然后找一个U盘，按照我们今天讲的详细步骤，完整地模拟一次对U盘的取证镜像全过程。

从执行 `blockdev --setro`，到使用 `dcfldd` 制作镜像、计算SM3哈希，再到最后填写 `Acquisition_Report.txt` 报告。亲手操作一遍，你会对取证镜像的严谨性和重要性有更深刻的理解。

今天的课就到这里，辛苦了。如果在练习中遇到任何问题，随时可以再来问我。

7.AI 问答过程

<https://www.doubao.com/thread/wc05302e2870e88b3>

8.github仓库地址

<https://github.com/zhu200511/2311/tree/main/330>